Week 7

A – Class Discussions

Even-based Processing

1 – What are the differences between time, request, and event-driven interactions?

	Time-driven	**Request-driven**	**Event-driven**
Trigger	A pre-scheduled time interval	A direct request from a client	An occurrence or change of state
Communication	N/A (based on internal schedules)	Synchronous (client waits for a response)	Asynchronous (producer doesn't wait for consumer)
Coupling	Loosely coupled (processes are independent)	Tightly coupled (client and server are dependent)	Loosely coupled (producer and consumer are decoupled)
Nature	Proactive, predictable	Reactive, on-demand	Reactive, real-time
Example	A script that runs at 2:00 AM every day to generate a report	A user clicking a button to get a product's price from a database	A sensor detects a change in temperature and automatically triggers an alert
Best For	Routine, batch processes where timing is predictable and not critical	Transactional systems require immediate, confirmed responses (e.g., payment processing)	Systems requiring real-time responsiveness and high scalability (e.g., IoT, notifications)

2 – What are the motivation and benefits of using event-based processing for IoT?

- The primary motivation for using event-based processing in IoT is to enable **real-time responsiveness and scalability** in a system with many distributed and diverse devices.
- IoT devices, like sensors and actuators, generate a continuous stream of data.
- Thus, event-based architecture is a natural fit for this environment because it focuses on reacting to specific occurrences (events) as they happen.
- The key benefits include:
  - **Real-time responsiveness**:
    - IoT applications often require immediate action based on sensor data.
    - For example, a fire alarm system needs to trigger an alert the moment smoke is detected, not after a scheduled check. Event-based processing enables devices to react instantly to changes in state.
  - **Scalability**:
    - IoT systems can involve millions of devices.
    - Event-driven architectures are highly scalable because components are loosely coupled.
    - Adding a new device or a new service to process a specific type of event doesn't require modifying existing parts of the system.
  - **Loose coupling**:
    - The producer of an event (e.g., a temperature sensor) doesn't need to know which services they are listening to for that event.
    - This allows different components to be developed, deployed, and updated independently, improving flexibility and resilience.
  - **Resilience and fault tolerance**:
    - If a consumer service fails, the events can be stored in an event broker and processed once the service comes back online.
    - This prevents data loss and ensures the system remains operational even if some components are temporarily unavailable.
  - **Efficient resource usage**:

    - Devices only communicate when an event occurs, rather than constantly polling for updates.
    - This reduces network traffic and conserves power on resource-constrained IoT devices.

3 – What are the challenges of event-based processing for IoT?

- **Increased complexity**:
    - The distributed and asynchronous nature of event-based systems can make them more complex to design, implement, and manage than traditional architectures.
    - Managing the lifecycle of many different event types and ensuring they are processed correctly can be daunting.

- **Debugging and monitoring difficulties**:
    - Troubleshooting problems in an event-driven system is challenging. Because events can trigger a cascade of actions across multiple decoupled services, tracing an event from its origin to its final effect requires specialized tools and a deep understanding of the system's architecture.

- **Event ordering and consistency**:
    - In a distributed system, events may not always arrive in the order they were sent.
    - Ensuring that events are processed in the correct sequence to maintain data consistency across all services can be a significant challenge.
    - This is particularly critical for applications where the order of events matters (e.g., a device on/off sequence).

- **Data heterogeneity**:
    - IoT devices are incredibly diverse, with different protocols, data formats, and communication standards.

- Processing events from such a wide variety of sources and converting them into a consistent format for consumption can be a major integration hurdle.

Complex Event Processing

1 - Why do we use Complex Event Processing (CEP)?

- To transform a deluge of raw, low-level data from multiple sources into actionable, high-level business insights in real time.
- It goes beyond simple event processing by analyzing and correlating seemingly unrelated events to detect meaningful patterns, threats, or opportunities that a single event would not reveal.

2 - What is the relationship between MQTT and CEP?

- MQTT (Message Queuing Telemetry Transport) is an asynchronous, publish/subscribe messaging protocol commonly used in IoT environments.
- Its lightweight nature and low power consumption make it ideal for resource-constrained devices like sensors.
- The relationship between MQTT and CEP is that MQTT acts as a primary event producer or data ingestion layer for a CEP system.

    - ❖ MQTT's Role: IoT devices use MQTT to publish simple, individual events (e.g., a temperature reading, a motion sensor trigger) to a central broker. These events are the raw ingredients for the CEP engine.

    - ❖ CEP's Role: The CEP engine subscribes to the topics on the MQTT broker, ingesting the stream of events. It then analyzes these events in real time to detect complex patterns.

- To sum up, MQTT handles the efficient and reliable transport of data from the edge to the central system, while CEP handles the intelligent analysis of that data stream. They work together to create complete, real-time event-driven architecture.

3 - What are the roles of real-time data enrichment, time series statistical processing, machine learning and data preparation in CEP?

- Data Preparation (or Preprocessing):
    - This is the foundational step.
    - Raw events from sources like MQTT often arrive in various formats and may contain missing or erroneous data.
    - The role of data preparation is to clean, normalize, and transform this data into a consistent format that the CEP engine can understand.
        - This can involve filtering out irrelevant events, aggregating events from a single source, and structuring the data for efficient processing.

- Real-time Data Enrichment:
    - Once data is prepared, enrichment adds context to raw events as they happen.
    - An event might contain a device ID and a temperature reading, but that information is more valuable when enriched with contextual data, like the device's geographical location, its owner's information, or the model number.
    - This is done by joining the real-time event stream with static or slow-changing data from a database, allowing the CEP engine to make more informed decisions based on a richer dataset.

- Time Series Statistical Processing:
    - This involves applying statistical analysis to a sequence of data points indexed by time.
    - In CEP, this is used to identify trends, anomalies, and patterns within a single stream of events over a specific time window.
    - Techniques include calculating a moving average to detect a gradual temperature increase, identifying a sudden spike in sensor readings, or counting events within a set timeframe.

    - This is critical for detecting situations like a slow leak or an unusual surge in network activity.

- Machine Learning (ML):
    - ML models can be embedded within the CEP pipeline to perform more sophisticated real-time analysis.
    - Instead of relying on predefined, rule-based patterns, ML models can learn from historical data to detect complex, non-obvious patterns.

- Anomaly detection:
    - An ML model can be trained to recognize "normal" system behavior and flag any events or sequences that deviate from it, which is more effective than setting static thresholds.

- Predictive analytics:
    - A model can analyze an event stream to predict a future event, such as a component failure in a factory machine, enabling proactive maintenance.

- Dynamic rules:

ML can help to continuously adjust and refine the rules that the CEP engine uses to identify complex events.

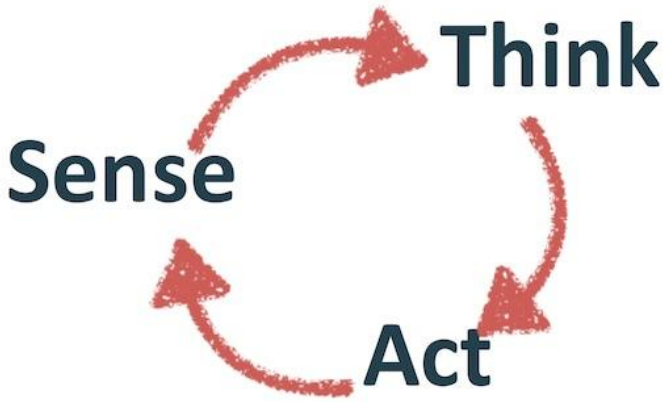
B – Group Activities

Do the following for 3 different event loops.

- Inputs needed
    - What sensors are needed if specific discrete data is needed?
    - What services are needed to be called for input?
    - What state needs to be updated by the event-loop?
- Output

    - What actuations are needed if specific actuation is needed?
    - What services are needed to be called for actuation?
- What processing is needed to make the output decisions?

To help you, think about Sense-Think-Act.



Produce pseudo-code: for each loop.

Event Loop 1 – Collision Avoidance

A - Inputs needed

- **Sensors**: LiDAR and radar sensors are needed to detect objects and measure their distance and velocity. Additionally, stereo cameras or vision systems are used to identify the type of object (e.g., pedestrian, another car, or a static object).

- **Services**: The car's mapping service is called to get information about the road ahead, while the GPS service provides the car's current location and speed.

- **State**: The loop updates the vehicle's state, including the current speed, acceleration, and the distance to the nearest detected obstacle.

B – Output

- **Actuations**: If an obstacle is detected within a critical distance, the brakes are actuated to slow down or stop the car, and the steering system is adjusted to swerve to avoid the collision if possible.

- **Services**: The vehicle control service is called to send commands to the braking and steering actuators.

- **Processing**: The system processes the sensor data to calculate the time to collision (TTC). This requires processing the distance and velocity of both the car and the detected object to determine if a collision is imminent.

C – Event Loop Pseudo Code

```
LOOP "Collision_Avoidance_Loop":
 SENSE:
  GET data from LiDAR, radar, and cameras
 THINK:
  IF obstacle_detected:
   CALCULATE time_to_collision (TTC)
   IF TTC < critical_threshold:
    DECIDE to brake and/or swerve
 ACT:
  IF DECISION is brake:
   CALL vehicle_control_service.apply_brakes()
  IF DECISION is swerve:
   CALL vehicle_control_service.adjust_steering()
END LOOP
```

Event Loop 2 – Traffic Ligh Recognition

A – Inputs needed

- **Sensors**: A **vision system** or camera is the primary sensor, capturing images of the road ahead to identify the color and state of traffic lights.

- **Services**: The **GPS service** is called to get the car's location, which helps the system anticipate and confirm the presence of traffic lights at intersections.

- **State**: The loop updates the car's **traffic state**, specifically whether a red or green light is ahead and how long it has been in that state.

B – Outputs

- Sensors: A vision system or camera is the primary sensor, capturing images of the road ahead to identify the color and state of traffic lights.

- Services: The GPS service is called to get the car's location, which helps the system anticipate and confirm the presence of traffic lights at intersections.

- State: The loop updates the car's traffic state, specifically whether a red or green light is ahead and how long it has been in that state.

C – Event Loop Pseudo Code

```
LOOP "Traffic_Light_Recognition_Loop":
 SENSE:
  GET real-time image feed from forward-facing camera
 THINK:
  PROCESS image to DETECT traffic light and its color
  IF traffic_light_color is RED:
   DECIDE to stop the car
  ELSE IF traffic_light_color is GREEN:
   DECIDE to proceed
 ACT:
  IF DECISION is stop:
   CALL vehicle_control_service.apply_brakes()
  IF DECISION is proceed:
   CALL vehicle_control_service.apply_throttle()
END LOOP
```

Even Loop 3 – Lane Keeping Assist

A – Inputs needed

- Sensors: Lane-tracking cameras or a vision system are used to identify the lane markings on the road.

- Services: The GPS service and mapping service are called to provide context on the type of road (e.g., highway, city street) and the expected lane curvature.

- State: The loop updates the car's lane position state, including its lateral distance from the center of the lane and its heading relative to the lane markings.

B – Outputs

- Actuations: The steering system is actuated with minor, continuous adjustments to keep the car centered in the lane.

- Services: The vehicle control service is called to send precise steering commands.

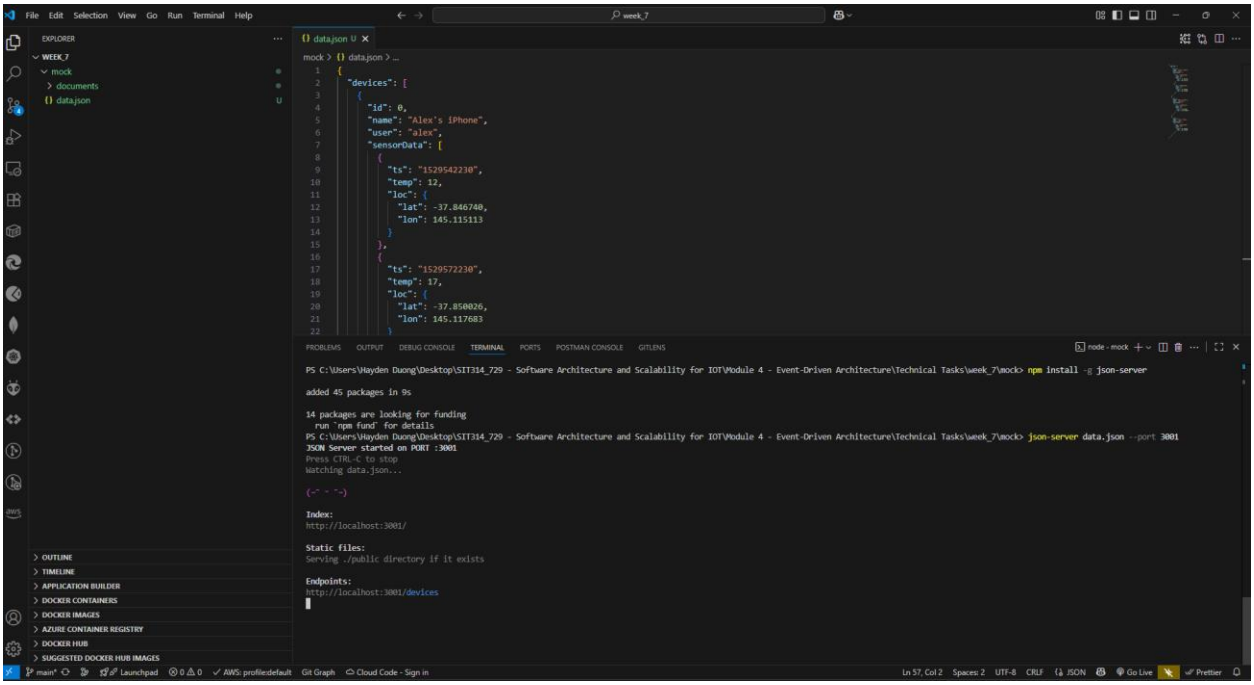
- Processing: This involves real-time image analysis to determine the position of the car relative to the lane lines. The system then uses

C – Event Loop Pseudo Code

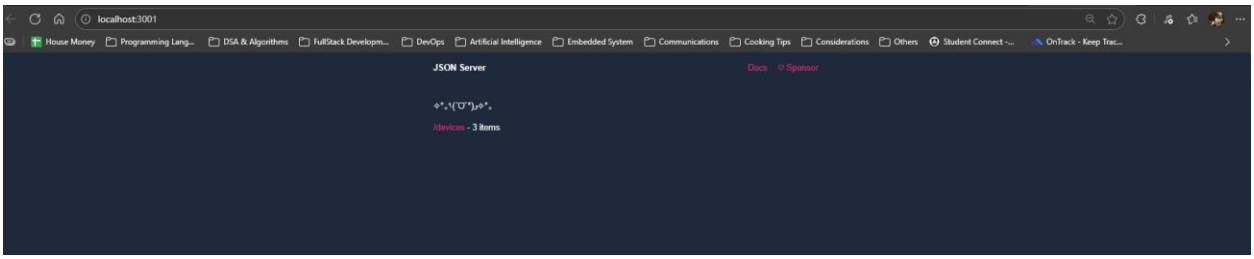
```
LOOP "Lane_Keeping_Assist_Loop":
 SENSE:
  GET image feed from lane-tracking cameras
 THINK:
  ANALYZE image to DETECT lane markings
  CALCULATE lateral_position_error from lane center
  IF lateral_position_error > tolerance:
   CALCULATE corrective_steering_angle
 ACT:
  IF corrective_steering_angle is needed:
   CALL vehicle_control_service.adjust_steering(corrective_steering_angle)
END LOOP
```

C – Technical Task

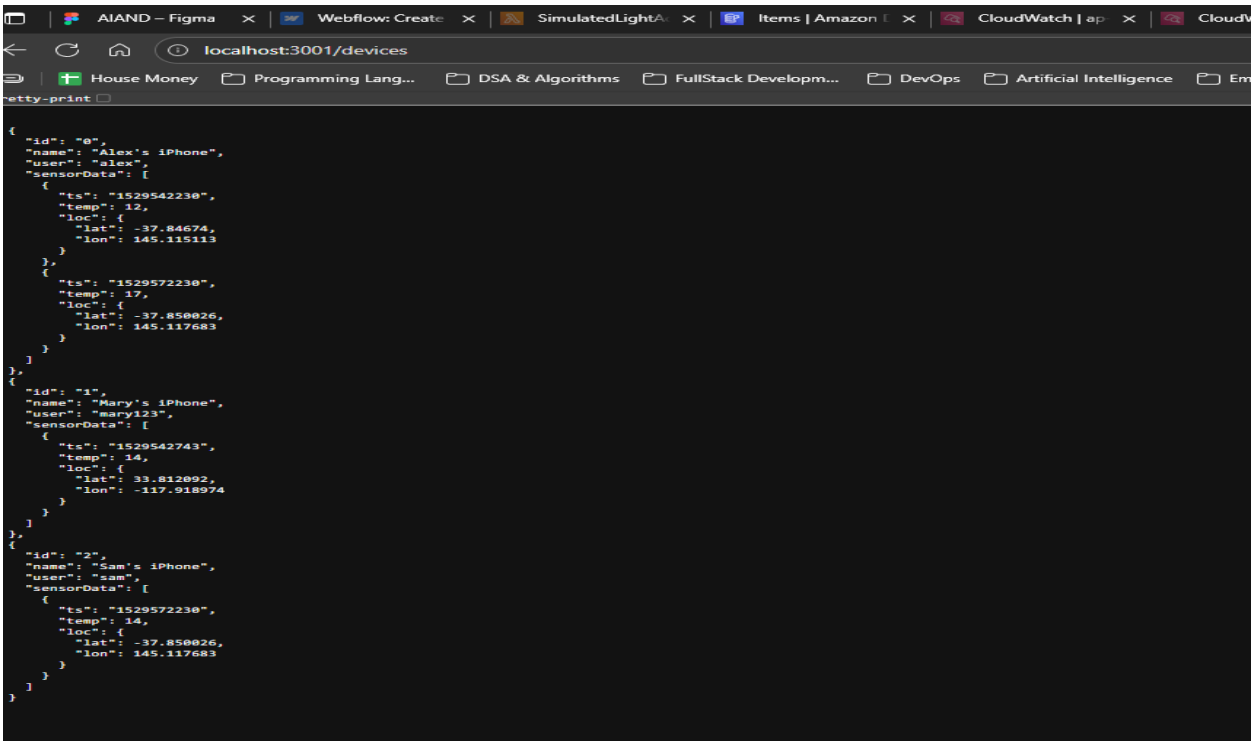
Screenshots



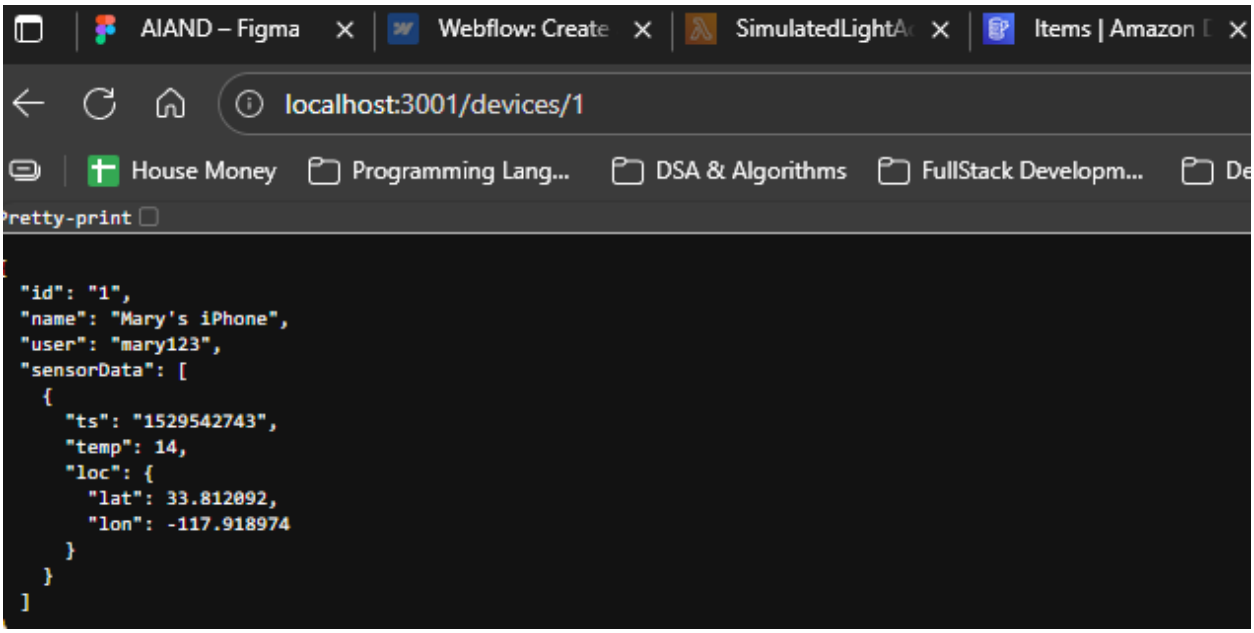
Picture 1 – Successfully run “json-server”.



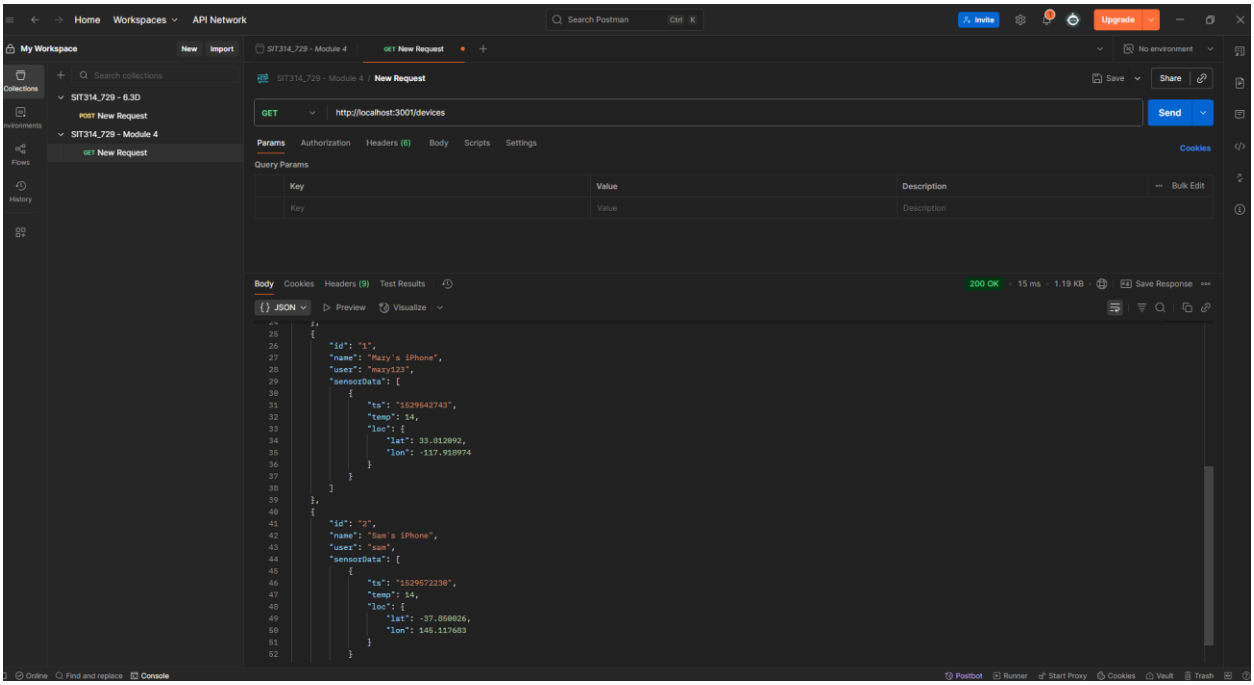
Picture 2 – Landing page of “json-server”.



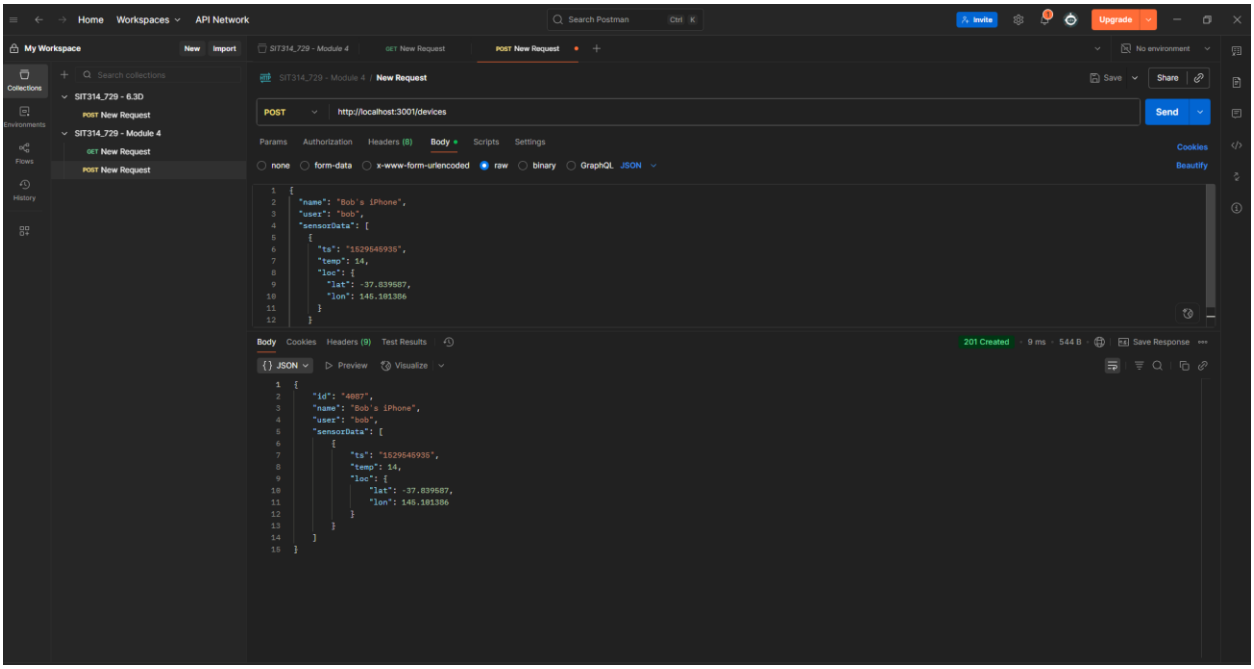
Picture 3 – Results gained from localhost:3001/devices



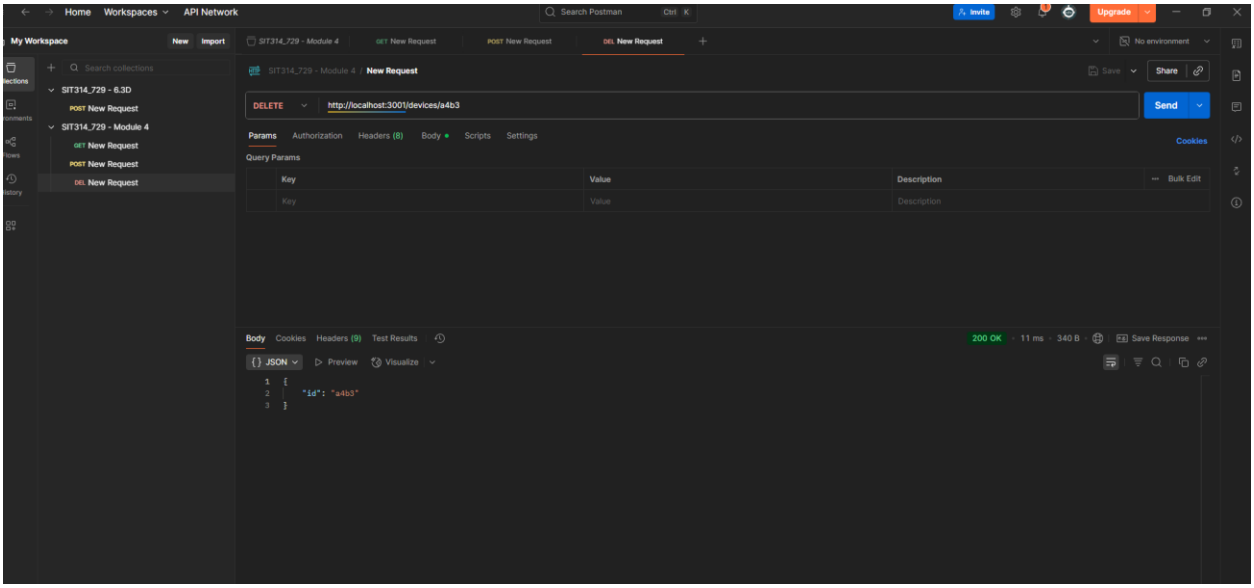
Picture 4 – Result from localhost:3001/devices/1



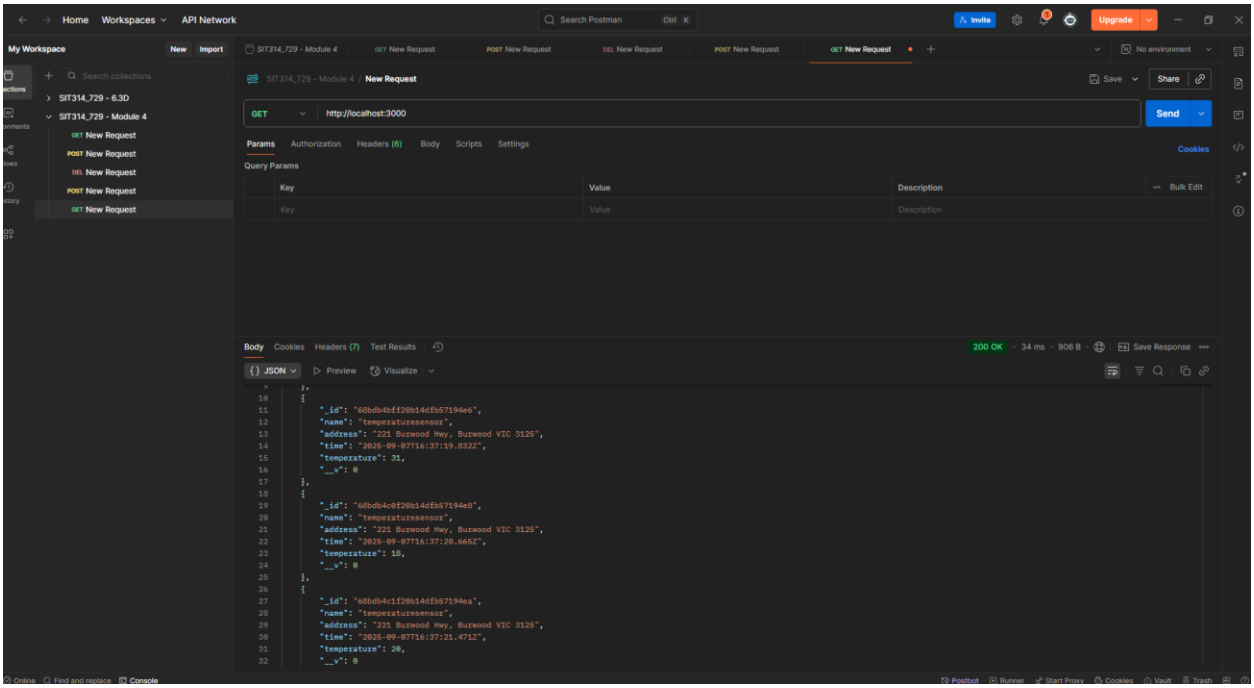
Picture 5 – Result from testing with Postman

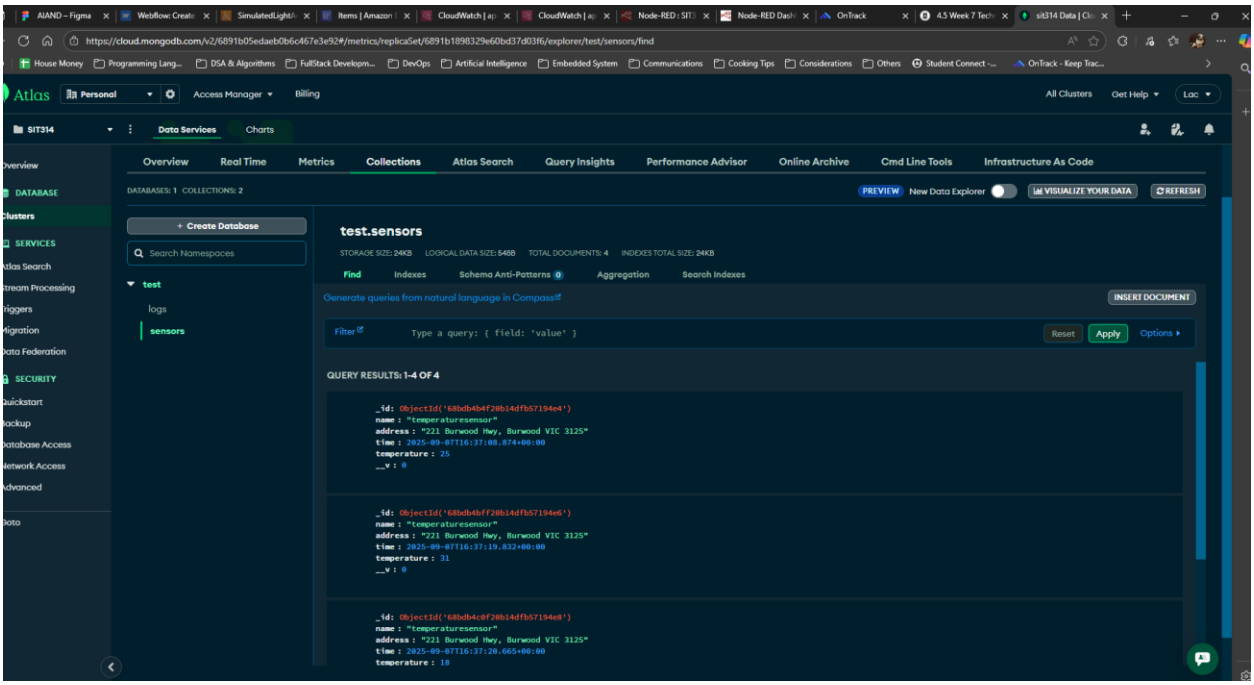


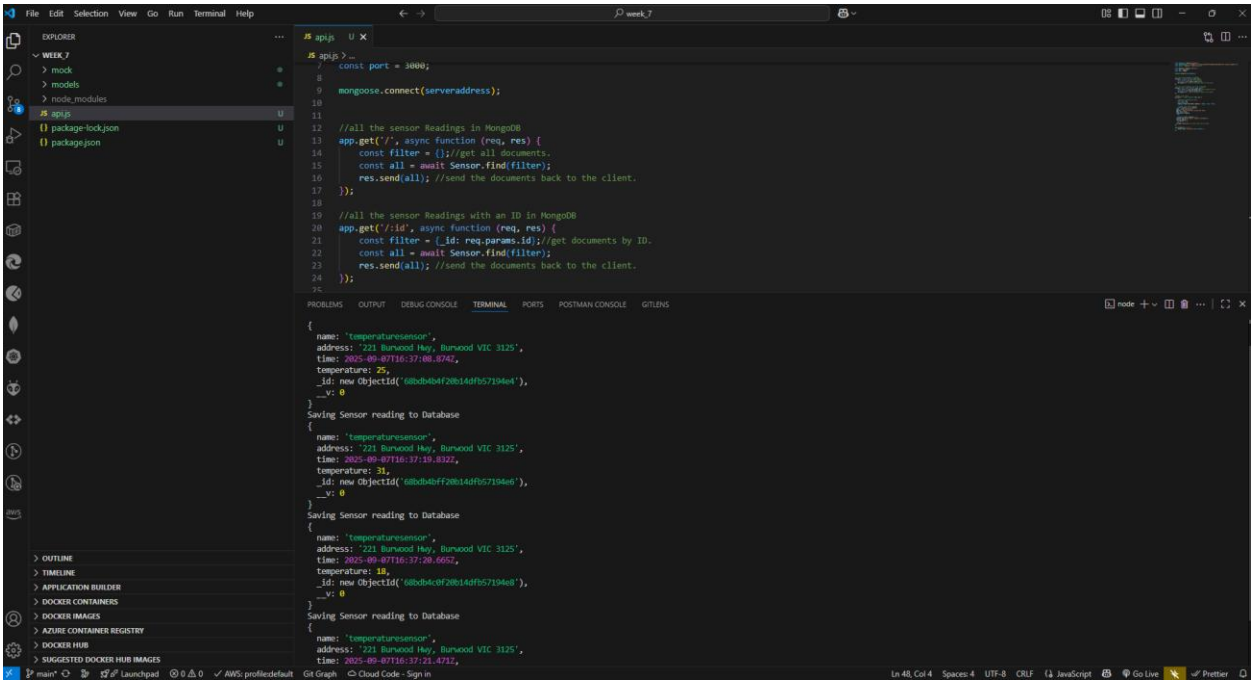
Picture 6 – Tested Post method with Postman



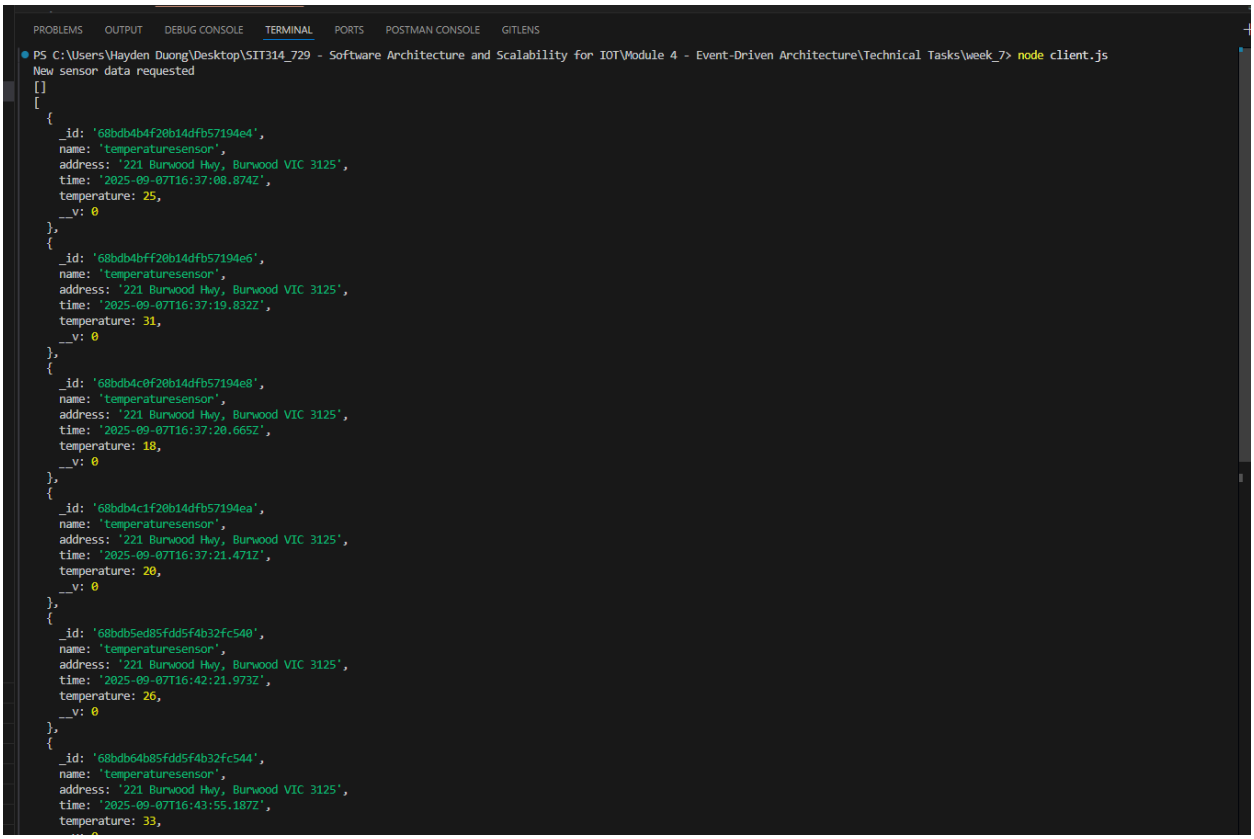
Picture 7 – Tested Delete method with Postman







Picture 8, 9, & 10 – Results gained from Step 10 – Task 7b



Picture 11 – Result gained from Step 1 – 5 of the final part

Week 8

A – Class Discussions

Micro-Service

1. What is microservice?

- A **microservice** is a small, independent service that performs a single, specific business function within a larger application. Each microservice has its own code, data, and dependencies, and can be developed, deployed, and scaled independently of other services. They communicate with each other through lightweight mechanisms, like APIs.

2. What is microservice architecture? And how does it relate to Separation of Concerns (SoC) principle?

- Microservices architecture is a software development approach that structures an application as a collection of loosely coupled microservices.
    - It's an alternative to the traditional monolithic architecture, where all components of an application are tightly integrated into a single unit.

- Microservices architecture is a prime example of the Separation of Concerns (SoC) principle.
    - SoC is a design principle that advises breaking a system into distinct sections, or "concerns," so that each section addresses a specific issue.
    - In a microservices architecture, each microservice represents a single concern, such as user authentication, payment processing, or product catalog management.
    - This separation ensures that a change to one service won't impact others, making the overall system more manageable, scalable, and resilient.

3. What are the similarities in the goals of microservices and IoT? And what are the challenges?

- Both microservices and the Internet of Things (IoT) aim to create systems that are modular, scalable, and distributed.
    - **Modularity**: Microservices break down large applications into small, manageable parts. Similarly, IoT systems are inherently modular, as they consist of numerous, distinct devices (sensors, actuators, etc.) that can be added or removed from the network.
    - **Scalability**: Microservices allow for individual services to be scaled up or down based on demand, rather than scaling the entire application. IoT requires similar scalability, as the number of connected devices can grow exponentially.

- **Distribution:** Both are distributed systems. Microservices are spread across a network of servers, while IoT devices are geographically distributed, often in remote locations.

- **Challenges**:
    - **Complexity**: Managing many microservices and IoT devices creates significant operational complexity. This includes managing a high volume of data, ensuring network stability, and monitoring the health of numerous distributed components.
    - **Data Management and Consistency**: In both microservices and IoT, data is decentralized. This makes it difficult to ensure data consistency across the entire system. Different services or devices may have their own databases, and maintaining synchronization can be a major challenge.
    - **Security**: A distributed architecture increases the attack surface. Each microservice and each IoT device is a potential point of entry for malicious actors. Securing inter-service communication and the vast number of devices is a complex task.
    - **Network Latency and Reliability**: Communication between microservices and between IoT devices can introduce latency and is subject to network failures. A single user request may require multiple calls across different services, and if any of those services or network connections fail, the entire operation could be impacted.

Domain-Driven Design

1. What is Domain-Driven Design (DDD) for microservices?

- Domain-Driven Design (DDD) is a software development approach that focuses on modeling software to match a specific business domain.

- For microservices, DDD is a powerful tool for defining clear, logical boundaries. It helps you break down a complex system into smaller, self-contained services that directly reflect the core business functions.
- Instead of organizing services around technical layers (like a database or user interface), DDD organizes them around the ubiquitous language of the business and its distinct subdomains, ensuring that each microservice has a clear, focused purpose.

2. How does each process within DDD work?

(i) Analyze domain

- The first step is to immerse oneself in the domain, which is the business area the software will provide (e.g., e-commerce, banking).
- This involves collaborating with domain experts to understand the business processes, rules, and terminology.
- The goal is to develop a ubiquitous language — a shared vocabulary used by both developers and business experts to talk about the system.

(ii) Define bounded contexts

- A bounded context is a logical boundary that encapsulates a specific part of the domain.
- Within each bounded context, the ubiquitous language and domain model are consistent and unambiguous.
    - For example, the term "customer" might have a different meaning in a "Sales" context (someone who has placed an order) versus a "Support" context (someone who has a support ticket). Identifying these contexts helps define the boundaries for individual microservices.

(iii) Define entities, aggregates, and services

- Once the bounded contexts are defined, you can model the components within them.

- ***Entities***: Objects that have a unique identity and can change over time. In a delivery service, a Delivery or a Drone would be an entity.

- ***Aggregates***: A cluster of related entities and value objects that are treated as a single unit.
    - An aggregate is a consistency boundary.
    - For instance, a Delivery aggregate could contain the Delivery entity itself, along with related entities like Package and value objects like Location.
    - All changes to the aggregate must go through its aggregate root, which is the primary entity (e.g., the Delivery entity) that ensures consistency.

- ***Services***: Stateless operations that span multiple entities or aggregates.
    - They perform business logic that doesn't naturally fit within a single entity.
    - For example, a SchedulerService might coordinate a Drone and a Delivery to assign a drone to a package delivery.

(iv) Identify microservices

- The final step is to map the DDD models to microservices.
- The most common approach is to align each bounded context with a single microservice.
- This ensures that each service has a clear, well-defined purpose and is responsible for its own domain model, data, and business logic.
- It also supports the principle of independent development and deployment.

3. How does the “Drone Delivery” use case illustrate the principles of DDD?

- Analyze domain:
    - The first step is to collaborate with domain experts (logistics managers, dispatchers) to understand the business.
    - The ubiquitous language would include terms like "Drone," "Package," "Delivery," "Scheduler," and "Supervisor."

- Define bounded contexts: The team would identify distinct subdomains that can operate independently. For example:
    - *Fleet Management Context*: Manages drones, their status, and maintenance.
    - ***Delivery Context***: Handles the lifecycle of a package delivery from order to completion.
    - ***Account Context***: Manages customer and business accounts.
    - ***Payment Context***: Processes payments for deliveries.
    - Each of these bounded contexts would become a separate microservice.

- **Define entities, aggregates, and services**: Within the Delivery Context, you would define your tactical patterns.
    - ***Entities***: A Delivery entity has an ID, a state (e.g., scheduled, in-flight, delivered), and a relationship to a drone.
    - ***Aggregates***: A Delivery aggregate would likely include the Delivery entity as its root, and a collection of Package entities. The Delivery root would enforce business rules, such as ensuring a package is not too heavy for the assigned drone.
    - ***Services***: A SchedulerService is a domain service that coordinates the Fleet Management and Delivery microservices. It would take a new delivery request and find a suitable drone, assigning it to the delivery.

- **Identify microservices**: Based on the bounded contexts and their aggregates, the team would design microservices such as the Fleet Management Service, Delivery Service, Account Service, and Payment Service. This structure ensures each service

is autonomous and loosely coupled, allowing for independent scaling and development.

B – Group Activities

The aim of the task is to identify the microservices needed to implement this system. Answer the following to design your system.

1 - What physical sensor and actuator infrastructure is needed in the supermarket?

- Sensors**:**
    - **Weight sensors** on shelves to track product levels.
    - **QR/barcode scanners** at checkouts and inventory locations to identify products.
    - **RFID readers** for real-time tracking of products and trolleys.
    - **Temperature sensors** in refrigerators and freezers to monitor food safety.
    - **Motion sensors** for automated lighting and security.

- Actuators**:**
    - **Automated conveyor belts** for restocking or delivery dispatch.
    - **Smart displays or digital price tags** to update prices and product information.
    - **Robotic arms or carts** for restocking shelves.

2 - What Edge Computing infrastructure is needed in the supermarket?

- **In-store server / gateway**: A local server is needed to process data from the sensors and actuators in real-time. This can handle tasks like immediate inventory checks and managing automated checkout processes.
- **Routers and network switches**: High-speed local network infrastructure is crucial to connect all sensors, actuators, and in-store devices.
- **Local database**: A local, lightweight database to store temporary data, such as real-time stock levels or recent transactions, for quick access by in-store microservices.

3 - What are the microservices needed for the system?

The system can be broken down into distinct microservices, each managing a specific business function.

- The system can be broken down into distinct microservices, each managing a specific business function.
- Inventory Microservice: Manages product stock levels, updates inventory counts from weight sensors, and triggers restocking events.
- Purchasing Microservice: Handles customer transactions, processes payments, and generates digital receipts.
- Restocking Microservice: Coordinates the automated restocking process, sends orders to suppliers, and manages in-store logistics.
- Staff Scheduling Microservice: Manages staff rosters, assigns tasks, and tracks employee hours.
- Delivery Microservice: Manages the last-mile delivery process, assigns orders to delivery drivers, and provides real-time tracking for customers.
- Analytics Microservice: Processes sales data, inventory trends, and customer behaviour to provide insights for business decisions.

4 - Thinking about 3 distinct microservices, sketch the pseudocode for the event processing loop.

A – Inventory Microservices

```
EVENT_LOOP:
   // Listen for events from sensors and other services
   event = listen_for_event()

   IF event.type == "PRODUCT_WEIGHT_CHANGE":
      product_id = event.payload.product_id
      new_weight = event.payload.new_weight
      // Update stock level based on weight change
      update_stock_level(product_id, new_weight)
      // Check if stock is below a critical threshold
      IF get_stock_level(product_id) < THRESHOLD:
         // Emit an event to trigger restocking
         emit_event("RESTOCK_NEEDED", {product_id: product_id})

   IF event.type == "RESTOCKING_COMPLETED":
      product_id = event.payload.product_id
      quantity_added = event.payload.quantity
      // Update stock and confirm restock
      update_stock_level(product_id, quantity_added)
```

```
        emit_event("INVENTORY_UPDATED", {product_id: product_id, new_level:
get_stock_level(product_id)})
```

B – Purchasing Microservices

```
EVENT_LOOP:
    // Listen for events from checkout scanners
    event = listen_for_event()

    IF event.type == "ITEM_SCANNED":
        product_id = event.payload.product_id
        // Get product details and add to current transaction
        product_details = get_product_details(product_id)
        add_item_to_cart(product_details)
        emit_event("CART_UPDATED", {cart_id: current_cart_id, item: product_details})

    IF event.type == "PAYMENT_RECEIVED":
        transaction_id = event.payload.transaction_id
        cart_details = get_cart_details(transaction_id)
        // Finalize transaction
        finalize_transaction(transaction_id, cart_details)
        // Emit event for inventory to deduct sold items
        emit_event("PRODUCTS_SOLD", {transaction_id: transaction_id, items:
cart_details.items})
```

C – Restocking Microservice

```
EVENT_LOOP:
    // Listen for events, primarily from the Inventory Microservice
    event = listen_for_event()

    IF event.type == "RESTOCK_NEEDED":
        product_id = event.payload.product_id
        // Check current supply and trigger a restock order
        IF check_supplier_stock(product_id) > 0:
            order = create_restock_order(product_id)
            send_order_to_supplier(order)
            emit_event("ORDER_SENT_TO_SUPPLIER", {order_id: order.id})
        ELSE:
            // Alert staff if supplier is out of stock
            emit_event("SUPPLIER_OUT_OF_STOCK", {product_id: product_id})

    IF event.type == "ORDER_RECEIVED_FROM_SUPPLIER":
        order_id = event.payload.order_id
        items_received = event.payload.items
```

```
    // Coordinate with in-store robotics or staff
    coordinate_in_store_restock(items_received)
    emit_event("RESTOCKING_COMPLETED", {items: items_received})
```

5 - Explore which microservices are critical to the performance of the application.

Purchasing Microservice:

- This is the direct interface with customers.
- If it's slow or fails, customers cannot buy products, directly impacting revenue and customer satisfaction.
- Latency here is unacceptable.

Inventory Microservice:

- This service manages the most critical business data: stock levels.
- Any delay or error could lead to incorrect stock information, causing out-of-stock items to appear available or vice versa, impacting both online and physical sales.

6- What microservices would you choose to scale, and what would be the triggers to scaling?

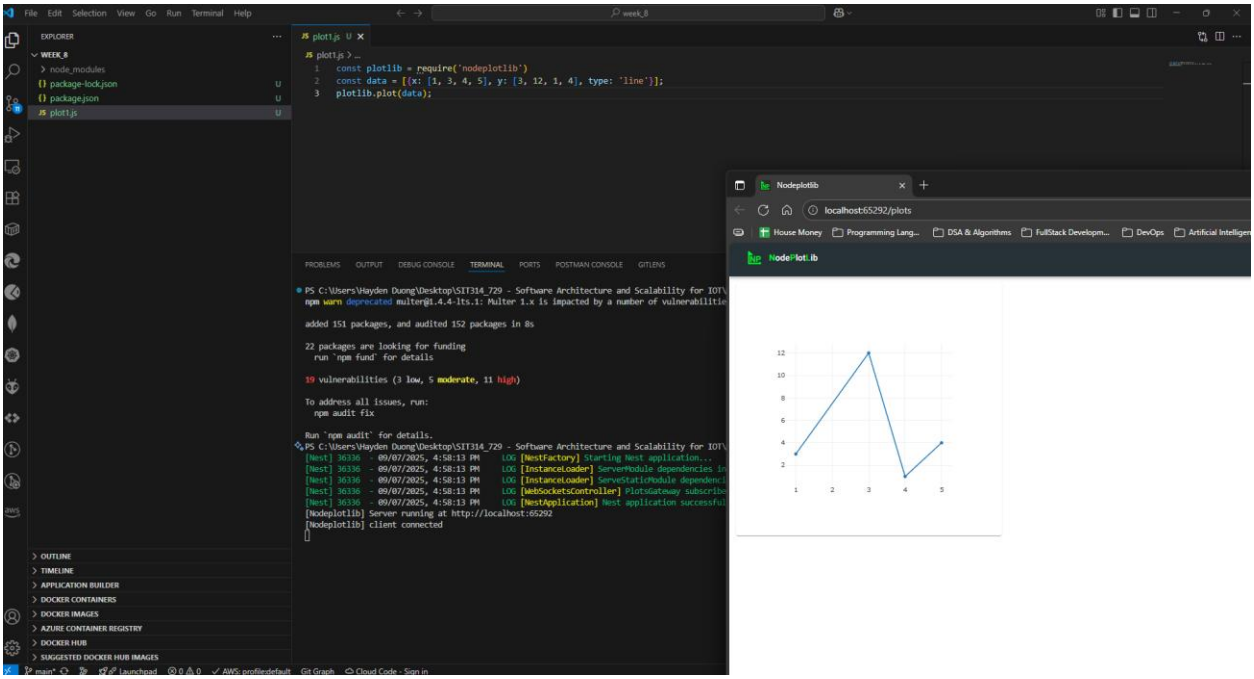
Purchasing Microservice: This service would need to scale horizontally to handle peak shopping hours, such as weekends, holidays, or during special promotions.

- **Triggers:**
    - **High transaction volume:** An increase in the number of concurrent checkouts or "Item Scanned" events would trigger new instances of microservice.
    - **Increased response time:** A spike in the average time to process a transaction would indicate system strain, triggering an autoscaling event.

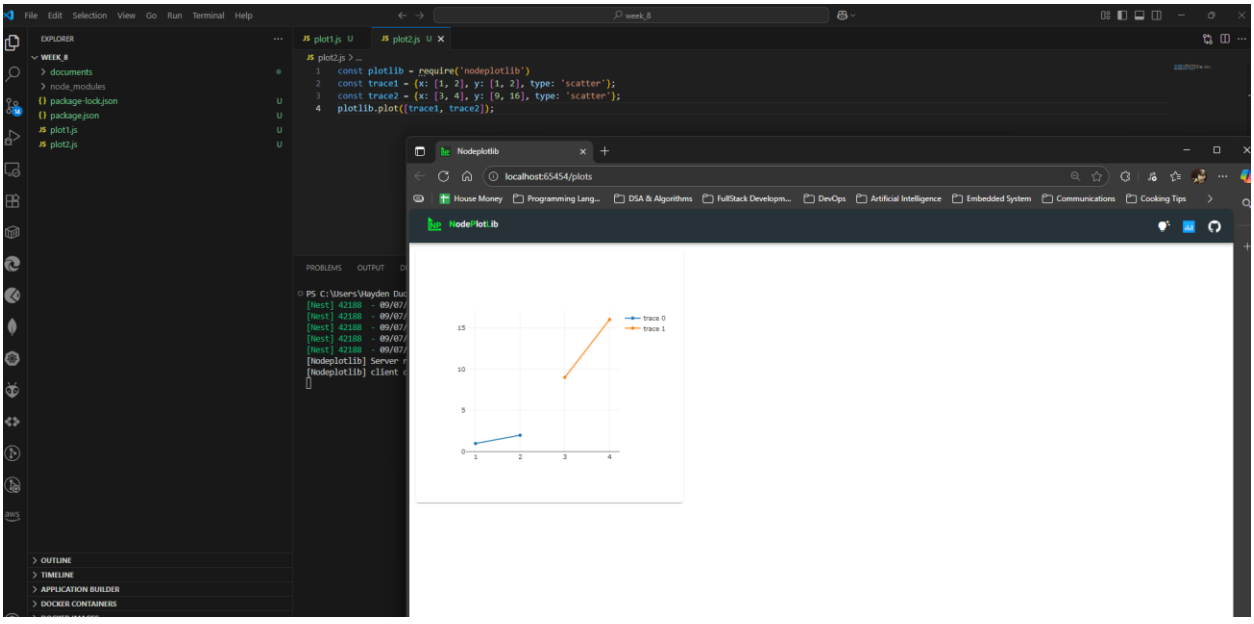
Inventory Microservice: This service needs to scale to handle the influx of data from a high number of sensors, particularly during restocking or high sales periods.

- **Triggers:**
    - **High event rate:** A surge in "PRODUCT_WEIGHT_CHANGE" or "PRODUCTS_SOLD" events from numerous shelves and checkouts would trigger scaling.
    - **CPU/Memory usage:** High resource consumption would indicate that the current instances are struggling to process the event stream, prompting the deployment of more instances.

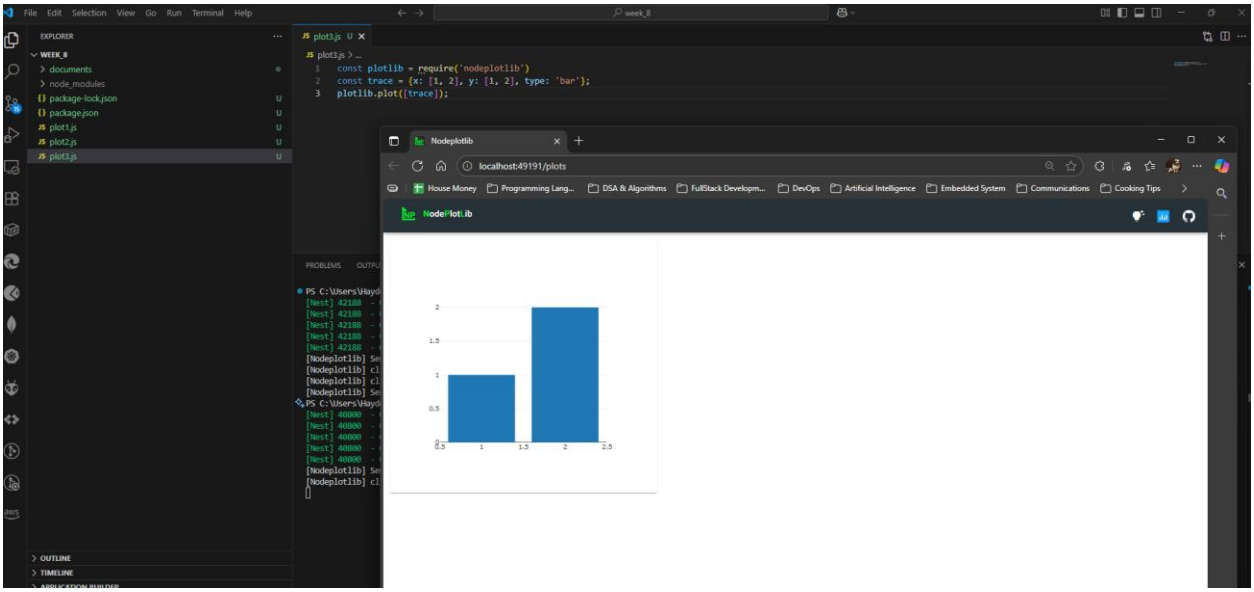
C – Technical Task



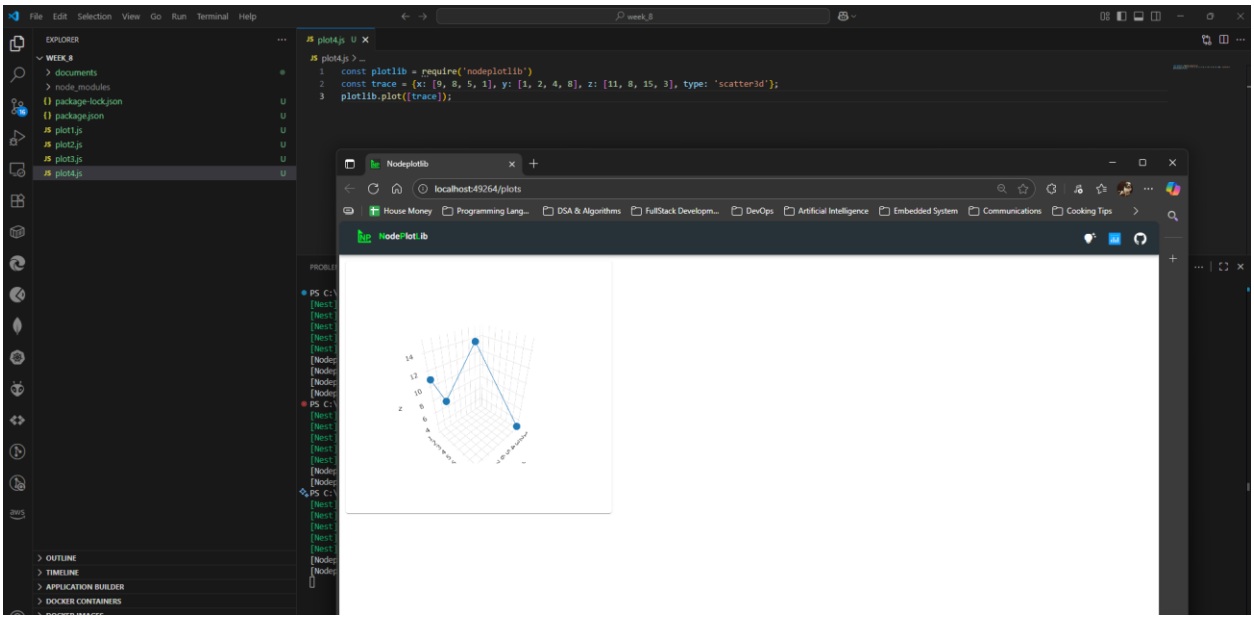
Picture 1 – Graph generated from “plot1.js”



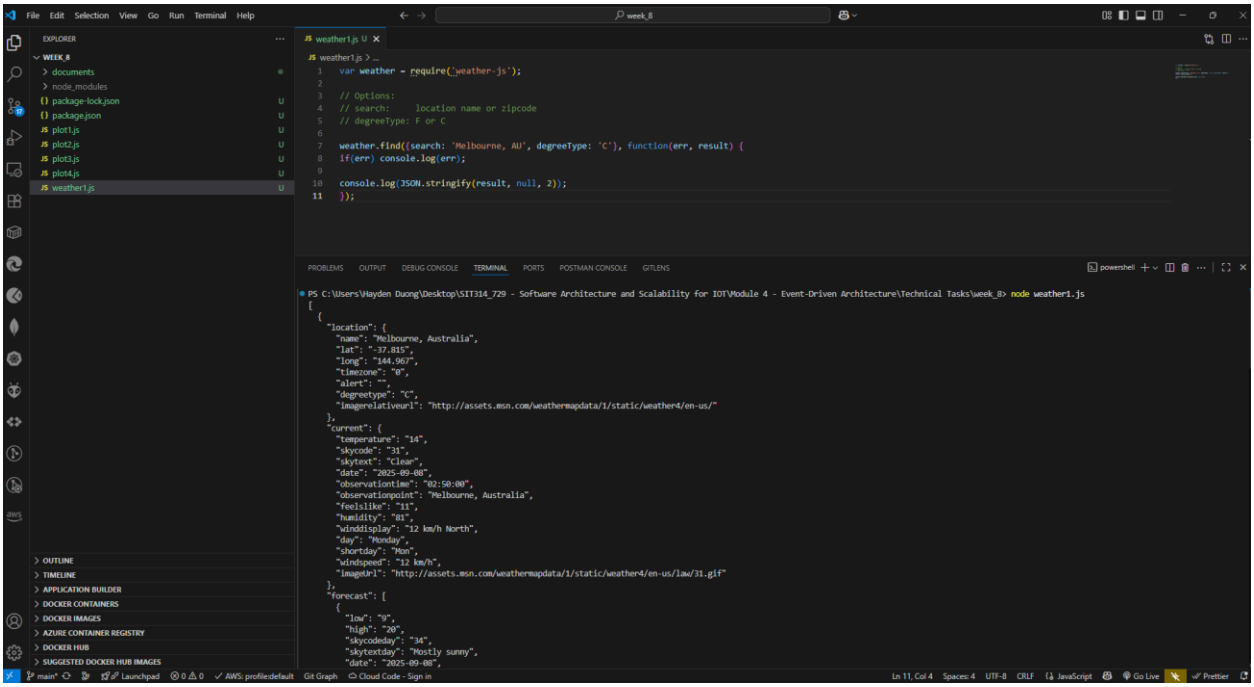
Picture 2 – Graph generated from “plot2.js”



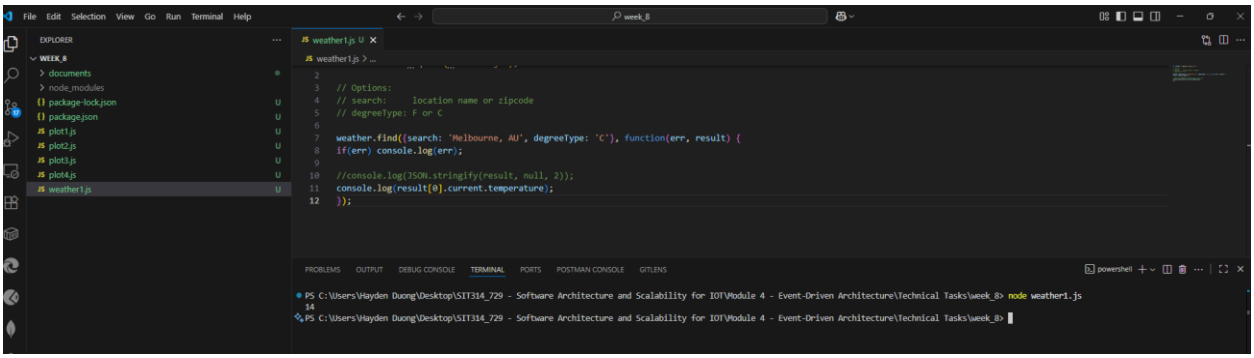
Picture 3 - Graph generated from “plot3.js”



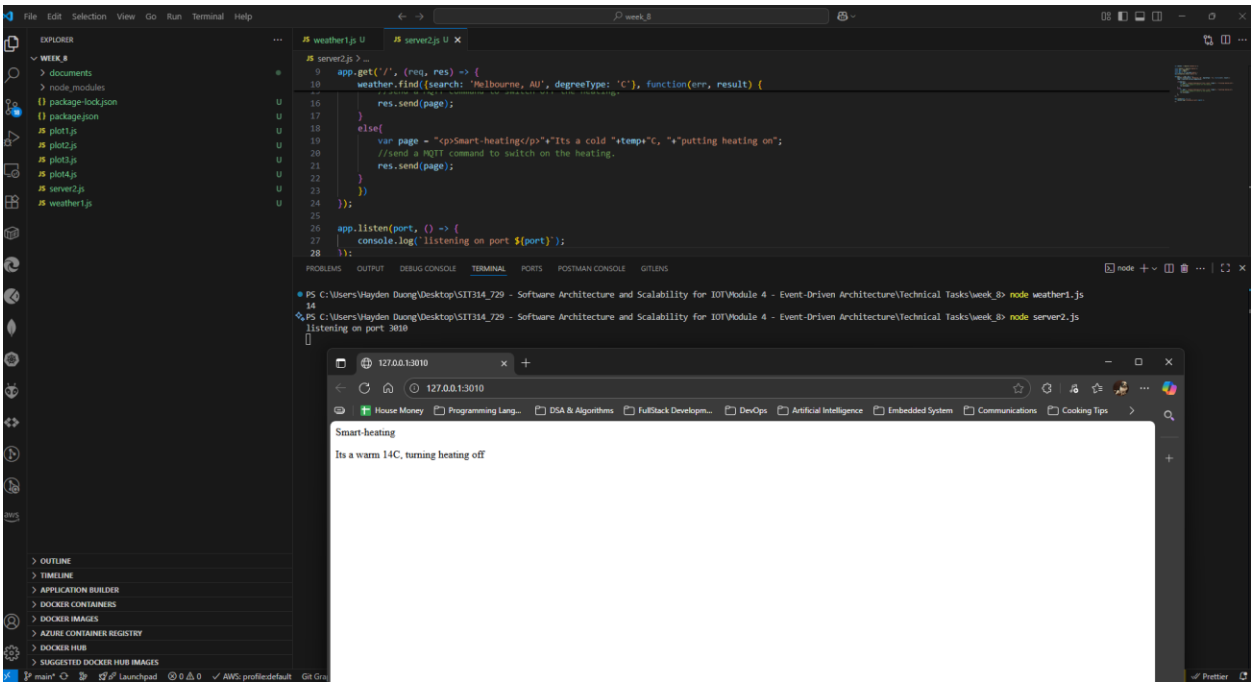
Picture 4 – Graph generated from “plot4.js”



Picture 5 – Result gained from running “weather1.js”



Picture 6 – Result gained from step 4 of “Accessing a data API using a module”



Picture 7 – Result gained from running “server2.js”

Module Summary:

This module introduced event-driven architecture, which focuses on asynchronous communication triggered by events. This approach differs from synchronous, request-driven interactions by being loosely coupled, scalable, and reactive in real-time. For IoT, event-driven processing is essential for handling vast streams of data, enabling real-time responsiveness, and ensuring resilience. We also explored Complex Event Processing (CEP), which analyzes and correlates multiple events to derive high-level insights. Additionally, the module covered microservices, a software architecture that breaks applications into small, independent services, and the role of Domain-Driven Design in defining these services.